

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 7

2 Artificial Intelligence (sbh11)

You have a supervised learning problem involving *classification*: a vector $\mathbf{x}$ is to be assigned to one of K classes. To do this you proceed in the usual way: you have a training set $\mathbf{s}$ containing m pairs $(\mathbf{x}_i, \mathbf{y}_i)$. However the labels $\mathbf{y}_i$ are now vectors in $\{0, 1\}^K$ containing a single 1 representing the target class. So for example if there are 5 classes and some $\mathbf{x}_i$ should be assigned to class 2 then $\mathbf{y}_i = (0, 1, 0, 0, 0)$. To do this, it is proposed that you use K neural networks. The i th network has parameters $\mathbf{w}_i$ and computes the function $h(\mathbf{w}_i, \mathbf{x})$. You may make no further assumptions regarding the function h .

NN output as probability

- (a) You aim to treat the output of the i th network as an estimate of the probability $\Pr(\mathbf{x} \in \text{class } i | \mathbf{x}, \mathbf{w})$ that $\mathbf{x}$ should be in the i th class, where $\mathbf{w}$ collects together all the K vectors $\mathbf{w}_1, \dots, \mathbf{w}_K$. It is proposed that to do this you should modify the setup described to compute

$$\Pr(\mathbf{x} \in \text{class } i | \mathbf{x}, \mathbf{w}) = \text{prob}(i, \mathbf{x}) = \frac{\exp(h(\mathbf{w}_i, \mathbf{x}))}{\sum_{j=1}^K \exp(h(\mathbf{w}_j, \mathbf{x}))}.$$

Explain why this modification is required, and how it achieves the stated aim. [4 marks]

Answer: We don't know anything about the range of h , which could include negative numbers. We need probabilities to sum to 1. The numerator makes positive values proportional to the network outputs (and $\exp$ has the bonus of being monotonic) and the denominator makes all outputs sum to 1.

Loss for NN output as probability

- (b) It is proposed that to train your networks, you should maximize the probability $\Pr(\mathbf{s} | \mathbf{w})$ that a given collection of weights would produce the data in $\mathbf{s}$. (You may consider the training inputs fixed.) Denote by $y_{i,j}$ the j th element of $\mathbf{y}_i$. Show that training can be achieved by minimizing

$$E(\mathbf{w}) = - \sum_{i=1}^m \sum_{j=1}^K y_{i,j} \log \text{prob}(j, \mathbf{x}_i).$$

State any assumptions that you make. [6 marks]

Answer: To maximise

$$\Pr(\mathbf{w}) = \prod_{i=1}^m \prod_{j=1}^K (\text{prob}(j, \mathbf{x}_i))^{y_{i,j}}$$

where we have assumed examples are iid and used the representation of the labels to select the required probability, we can take its log, then negate to get a minimization. The result follows immediately.

Modification of backpropagation

- (c) You have previously applied the backpropagation algorithm for training the

networks $h(\mathbf{w}_i, \mathbf{x})$ and as a result of this you know how to compute derivatives $\partial h(\mathbf{w}_i, \mathbf{x}) / \partial w_{i,j}$ where $w_{i,j}$ is the j th element of $\mathbf{w}_i$. Explain what further steps are necessary to use this knowledge to obtain derivatives of $E(\mathbf{w})$ with respect to the relevant weights. [10 marks]

Answer: The fundamental quantities needed are $\partial E(\mathbf{w}) / \partial w_{o,p}$. Plugging in what we have and writing

$$Z = \sum_{v=1}^K \exp(h(\mathbf{w}_v, \mathbf{x}_i))$$

gives

$$\begin{aligned} \frac{\partial E(\mathbf{w})}{\partial w_{o,p}} &= - \sum_{i=1}^m \sum_{j=1}^K y_{i,j} \frac{\partial}{\partial w_{o,p}} \log \left(\frac{\exp(h(\mathbf{w}_j, \mathbf{x}_i))}{Z} \right) \\ &= - \sum_{i=1}^m \sum_{j=1}^K y_{i,j} \frac{\partial}{\partial w_{o,p}} [h(\mathbf{w}_j, \mathbf{x}_i) - \log Z]. \end{aligned} \quad (1)$$

Also,

$$\begin{aligned} \frac{\partial \log Z}{\partial w_{o,p}} &= \frac{1}{Z} \frac{\partial Z}{\partial w_{o,p}} \\ &= \frac{1}{Z} \frac{\partial Z}{\partial w_{o,p}} \left(\sum_{v=1}^K \exp(h(\mathbf{w}_v, \mathbf{x}_i)) \right) \\ &= \frac{1}{Z} \frac{\partial Z}{\partial w_{o,p}} \exp(h(\mathbf{w}_o, \mathbf{x}_i)) \\ &= \frac{1}{Z} \exp(h(\mathbf{w}_o, \mathbf{x}_i)) \frac{\partial h(\mathbf{w}_o, \mathbf{x}_i)}{\partial w_{o,p}} \\ &= \text{prob}(o, \mathbf{x}_i) \frac{\partial h(\mathbf{w}_o, \mathbf{x}_i)}{\partial w_{o,p}}. \end{aligned} \quad (2)$$

The final chunk is

$$\frac{\partial h(\mathbf{w}_j, \mathbf{x}_i)}{\partial w_{o,p}} = \begin{cases} \frac{\partial h(\mathbf{w}_o, \mathbf{x}_i)}{\partial w_{o,p}} & \text{when } o = j \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

Combining Equations 1, 2 and 3 we have an expression depending only on the derivatives we already know, so gradient descent can be applied as usual.
